Java function | Deep Dive | Recursion | Method Overloading | java Day #11

functions in java. → Block of code.

```
{
  =
  =
}
```

Public static Void main(string [] arg) {
int i = 4;
int j = 5;
int result = sum (i, j); → Arguments (literals)
System.out.Print-In (result);
}

calling a function.

Boiler plate code. (all code)

```
Public static void main
        String [] arg {
  ——
  ——
}
```

Return type
↓
Datatype that is return by function.

Static int sum (int a, int b) {
int result = a + b;
return result;
}

Returntype → fname
Parameters.
return an output

Functions → i/p → [f^n 2] → o/p in Mathematicaly.

```
Sum (a, b) {
  ——
  ——
}
```

→ result is almost same.

Types of functions →

No I/P, No O/P
I/P, No O/P
No I/P, No O/P
I/P, O/P

```
Public class Demo {
Public static Void main (string [] args) {
//functions in java
~~Hello~~ get ();
    SayHello ("Rohit");  Argument
    System.Out.Print-In (get-number ());
    System.Out.Print-In (multiply (2, 4));
}
static Void get () {   // No I/P / No O/P
    System.Out.Print-In ("Hello");
    return;
}
Static Void say Hello (string name) {   // Number of parameters can be anything,
    System.out.Print-In ("Hello" + name);
    return;
}
Static ~~void~~ int get number () {   // No I/P, O/P
    return 10;
}
Static int multiply (int a, int b) {
    // return (a+b);
```

empty
empty → kuch return nhi karta.
I/P
parameters.

output
- Hello
- Hello Rohit
- 10
- 8

I P. No O P

function Overloading →

eg →

```
Public class Demo2 {
    Public static void main (string [] args) {
    int x = sum (2,3);
    System.Out.Print-In (x);
    int y = sum (5,6);
    System.Out.Print In (y);
    int c = sum (6,7);
    System.out.Print In (c);
  }
  static int sum (int a, int b) {
    return (a+b);
  }
}
```

2num

output
• 5
• 11
• 13

//add 3numbers.

```
    int z = sum2 (3,4,8);
    System.Out.Print-In (z);
  }
  static int sum2 (int a, int b, int c) {
    return (a+b+c);
  }
}.
```

different number of parameters

• 15

- function overloading → ~~java~~ we use in java same function name. eg → sum
  we can use function with same if we want to perform the same task. in java. lik subst, addition, multipl
- you dont need to create another function for this.
- you can use different types of data types.

eg. double,

different data types of parameters

```
static int sum (double a, double b) {
    return (int) (a+b);
  }.
```

```
static void greet (string name, int age) {
  System.out.Print-In ("Hi" + name + "your age is" + age);
}
static void greet (~~string name~~ int age, string name) {
  System.Out.Print In ("Hi" + name + "your age is" + age);
}
```

} under main function start

```
greet (name, age);
greet (age, name);
```

} even use to this type in jav
Ordering of parameters

```
Static void fun ( ) // with No Parameters
System.Out.Println ("Hello");
}
Static int fun( ) {
    System.out.println ("Hello");
    returns;
}.
```

There is Compiler errore.

Because of No Parameters. Compiler will not understand what i do you can use different function.

Chaining of functions.

```
Public static void main (string [ ] args) {
  A( );
}
Public static void A (string [ ] args) {
  B( );
}
void B (string[] args ) {
  C( ); // System.out.Print.In ("Hello");
}.
```

A to B to C

chaining

```
Public static void main (string [ ] args) {
  A ( );
  System.Out.Print.In ("Hello Sachin");
}
void A( ——— ) {
  B ( );
  System.out.Print.In ("Sachin");
}
void B ( ——— ) {
  System.out.Print.In ("Hello Deepak");
```

output

- Hello Deepak.
- Sachin
- Hello Sachin.

```
Public class Demo2 {
  Public static void main (String [ ] args) {
  // chainning of function.
  fun1();
  System.out.Print In ("Hello");
  }
Static void fun1 () {
  fun2 ();
  System.Out.Print In ("Deepak");
  }
Static void fun2 () {
  fun3 ();
  System.out.Print In ("Hii");
}
static void fun3 () {
  System.out.Print In ("Sachin");
}.
```

output

- Sachin.
- Hi
- Deepak
- Hello.

• Scope of A Variable ⟶.

```
Public class Demo2 {
  Public static void main (string [ ] args ) {
  // scope of A variable.
 int a=5; //Local Scope
 int b=6;
 if (a==6) {
   int j=7;
   System.Out.Print.In (j);
 }
 System.out.Print In (a+", "+b);
 fun ();
}
static void fun () {
 int x=4; /Local scope.
 int y=5;
 System. out.Print In (x+", "+y);
}
```

under the code blocks you Can not Access these any other places.

Global Variable Uses. → Use Anywhere in class.

```
Public class Demo4{
 Static String name="Sachin"; //Global variable
 Public static void main (String [] args){
  int x=4; // Local variable
  int y=5;
  if (x==4){
    int j=7;
    System.Out.Print-In (j);
  }
  System.Out.Print-In (x+", "+y);
  System.Out.Print-In (name); // Global.
  fun ();
 }
 Static void fun (){
  int a=4; Local variable
  int y=5; Local variable
  System.Out.Print-In (name); // Global
  System.out.print-In (a+", "+b);
 }
```

• Output
• 4, 5
• Sachin.
~~• 4, 5~~
• Sachin
• 4, 5

• Recursion → A function which Call itself.

```
Public static void main (String [] args){
  int n=5;
  Print-num (n);
}.
Void Print-num (int n){
  if (n==0) return; Base case
  print (n-1); // Recursive call.
  System.Out.Print-In (n);
}.
```

Print(5)
↳ Printnum(4)
↳ Printnum(3)
↳ Printnum(2)
↳ Print-num(1) Base Case
↳ Print-num(0)

```
main (...){
 A ();
}

void A (){
 A ();
}
```

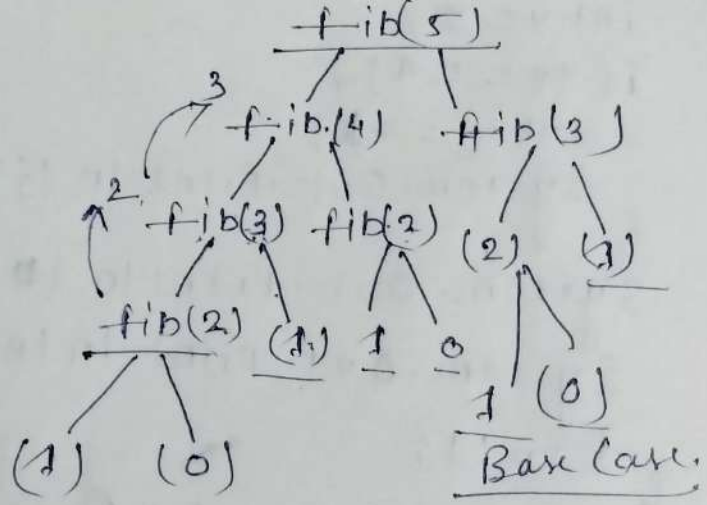
output - 1
2
3
4
5

fibonacci number.

1, 1, 2, 3, 5, 8, 13 . . . .

nth fibonacci.

Mathematicaly

fib(n) = fib(n-1) + fib(n-2);

```
int fib(n){
    if(n==0)|| n==1) return 1;
    int x = fib(n-1);
    int y = fib(n-2);
    return x+y;
}
```